Modul Bank Accounts

Seluruh test case pada docs/qa/test-cases/bank-accounts.md. Eksekusi dilakukan lewat request HTTP dengan cookie sesi asli dan diverifikasi ke database — bukan diasumsikan dari kode. Perilaku visual antarmuka React tidak termasuk cakupan dan karena itu laporan ini tidak memuat bukti layar.

38	37	9	3
Test case dijalankan	Akhirnya lolos	Gagal di percobaan pertama	Bug diperbaiki

38 test case dirancang untuk modul ini; 37 lolos, 1 gagal, 0 tidak dijalankan dan sengaja tidak dibulatkan menjadi lolos.

5 temuan tercatat (BUG-BA-01, BUG-BA-02, BUG-BA-03, BUG-BA-04, BUG-BA-05). Rincian sebab dan cakupannya ada di blok temuan di bawah.

Bug yang ditemukan & diperbaiki

BUG-BA-01 · Menghapus rekening menghapus pembayaran invoice tanpa memperbaiki status invoice — SUDAH DIPERBAIKI

SIFAT	Data integrity / kehilangan data finansial diam-diam.
MENGGAGALKAN	C02 (lihat juga N14).
GEJALA	Setelah `DELETE /c/qa-dua/bank-accounts/4`, tabel `payments` kosong (pembayaran 400.000 atas invoice `001/INV/QD-UB/II/2026` lenyap), tetapi `invoices.id=1` tetap `status=partially_paid` dengan `total_amount=1.000.000`. Invoice sekarang berstatus "dibayar sebagian" padahal `amount_paid` = 0. Tidak ada peringatan apa pun sebelum menghapus.
SEBAB	`app/Http/Controllers/BankAccountController.php` — `destroy()` memanggil `$bankAccount->delete()` tanpa guard apa pun; penghapusan `payments` terjadi di level DB lewat FK `onDelete cascade` (`database/migrations/2023_01_01_000008_create_payments_table.php`), sehingga event model Laravel tidak jalan dan `Invoice::updateStatus()` tidak pernah dipanggil. Perilaku cascade-nya sendiri memang didokumentasikan (`docs/module/bank-accounts.md`, bagian "Invarian & Jebakan"), tapi konsekuensi status invoice tidak.
CAKUPAN	Setiap penghapusan rekening yang pernah menerima pembayaran invoice. Makin banyak invoice yang dibayar lewat rekening itu, makin banyak invoice yang statusnya jadi bohong. Berdampak ke laporan piutang & Laba Rugi.
PERBAIKAN	Di `destroy()`, sebelum delete: kumpulkan `invoice_id` dari `$bankAccount->payments`, hapus akun, lalu panggil `updateStatus()` untuk tiap invoice tersebut — semuanya dalam satu `DB::transaction`. Lebih aman lagi: tolak penghapusan bila rekening masih punya `payments`, dan minta user memindahkan/menghapus pembayarannya lebih dulu (konsisten dengan pola *reassign* di modul Kategori).

BUG-BA-02 · Parameter `sort` & `direction` tab Transaksi tidak divalidasi → HTTP 500 + kebocoran SQL — BELUM DIPERBAIKI

SIFAT	Robustness + information disclosure.
MENGGAGALKAN	A11, A12, A13.
GEJALA	- `GET /c/qa-dua/bank-accounts/transactions?account=1&sort=DROP` → 500, body memuat `SQLSTATE[42S22]: Column not found: 1054 Unknown column 'DROP' in 'order clause' (Connection: tenant, Host: 127.0.0.1, Port: 3306, Database: tenant_qa-dua, SQL: select * from bank_transactions where ...)` — nama host, port, dan nama database tenant ikut terkirim. - `?direction=XYZ` → 500 `InvalidArgumentException: Order direction must be "asc" or "desc".`
SEBAB	`app/Http/Controllers/BankTransactionController.php:33-35, 49` — `$sort` dan `$direction` diambil mentah dari request lalu langsung dipakai `->orderBy($sort, $direction)`. Bandingkan dengan `indexPayments()` di file yang sama yang sudah memakai `match()` whitelist (terbukti aman di N16). Tidak ada injeksi SQL (Laravel meng-*quote* identifier — dibuktikan A12), tapi penanganan errornya salah.
CAKUPAN	Endpoint tab Transaksi. Kebocoran detail koneksi hanya terjadi bila `APP_DEBUG=true`; di lingkungan uji ini `APP_ENV=local`. Di production gejalanya menjadi 500 polos — tetap cacat.

BUG-BA-03 · `per_page` tab Transaksi tanpa batas atas — BELUM DIPERBAIKI

SIFAT	Availability / resource exhaustion.
MENGGAGALKAN	A14.
GEJALA	`GET /bank-accounts/transactions?account=1&per_page=100000` dijawab 200 dengan `"per_page":100000`. Nilai apa pun diterima.
SEBAB	`BankTransactionController.php:32` — `$perPage = (int) $request->input('per_page', 15);` lalu langsung `->paginate($perPage)`, tanpa `min()`/whitelist.
CAKUPAN	Tab Transaksi (dan pola yang sama perlu dicek di `indexPayments()`). Pada tenant dengan ratusan ribu transaksi, satu request bisa menarik semuanya ke memori.

BUG-BA-04 · Hapus rekening yang punya `loan_payments` melempar 500 mentah — SUDAH DIPERBAIKI

SIFAT	Error handling / UX.
MENGGAGALKAN	A15.
GEJALA	`DELETE /c/qa-dua/bank-accounts/6` → HTTP 500 `SQLSTATE[23000]: Integrity constraint violation: 1451 Cannot delete or update a parent row: a foreign key constraint fails (tenant_qa-dua.loan_payments, CONSTRAINT loan_payments_bank_account_id_foreign FOREIGN KEY (bank_account_id) REFERENCES bank_accounts (id))`. Data tetap aman (akun tidak terhapus), tapi user hanya melihat layar error.
SEBAB	`BankAccountController::destroy()` tidak mengecek `$bankAccount->loanPayments()->exists()`. FK `loan_payments.bank_account_id` memang sengaja tanpa cascade (`database/migrations/2025_11_26_083021_create_loan_payments_table.php`), dan hal ini sudah dicatat di `docs/module/bank-accounts.md`, tapi tidak pernah diterjemahkan menjadi guard di controller.
CAKUPAN	Semua rekening yang pernah dipakai membayar cicilan pinjaman.
PERBAIKAN	Tambahkan guard sebelum delete: `if ($bankAccount->loanPayments()->exists()) { return back()->withErrors(['delete' => 'Rekening ini dipakai pada pembayaran pinjaman dan tidak dapat dihapus.']); }` — sejalan dengan gaya pesan guard di `TransactionCategoryController::destroy()`.

BUG-BA-05 · `/api/bank-accounts` tidak digate `can:view bank-accounts`	SUDAH DIPERBAIKI
SIFAT	Information disclosure (ringan) / inkonsistensi otorisasi.
MENGGAGALKAN	A17.
GEJALA	`staff@gmail.com` mendapat 403 di `/c/qa-dua/bank-accounts` tetapi 200 di `/c/qa-dua/api/bank-accounts`, yang mengembalikan daftar lengkap rekening perusahaan (`QA Rekening Utama (BCA)`, `QA Rekening Pinjaman 2 (BRI)`, `QA Manager Acc (CIMB)`, `QA Rekening Kedua (Mandiri)`). Saldo & nomor rekening tidak ikut.
SEBAB	`routes/web.php:108-116` — closure `api.bank-accounts` hanya berada di dalam grup `c/{company}` (auth + tenancy), tanpa `can:` apa pun.
CAKUPAN	Setiap anggota perusahaan yang login. Kemungkinan disengaja: staff punya `create invoices`/`create receivables` sehingga butuh picker rekening; comment di `routes/web.php:319` menunjukkan pola serupa memang dipertahankan untuk user cash-flow. Dicatat agar jadi keputusan sadar, bukan kelalaian.
PERBAIKAN	Gate dengan izin gabungan yang mencerminkan pemakaian sebenarnya, mis. `->middleware('can:any,view bank-accounts/create invoices/create receivables')` (atau Gate kustom), lalu dokumentasikan keputusannya di `docs/module/bank-accounts.md`. ---

Hasil per test case

percobaan gagal (bug aplikasi) gagal karena kendala alat uji, bukan bug percobaan lolos

ID	SKENARIO	PERCOBAAN	CATATAN: KENAPA GAGAL & APA PERBAIKANNYA	STATUS
A. KONDISI NORMAL				
N01	**Buat rekening baru**		HTTP 302; DB: `id=1 account_number=1000000001 initial_balance=1000000`	Pass
N02	**Buat rekening kedua dengan `initial_balance=0` (batas bawah `min:0`)**		HTTP 302; DB `id=2 initial_balance=0`	Pass
N03	**Saldo adalah accessor computed, bukan kolom**		Kolom yang ada: `id, account_name, account_number, bank_name, branch, initial_balance, created_at, updated_at` — tidak ada `balance`	Pass
N04	**Saldo bergerak setelah transaksi credit & debit**		Prop Inertia: `{"id":1,...,"initial_balance":1000000,"balance":1300000}`. DB `bank_accounts` tetap tanpa kolom saldo	Pass
N05	**Ubah `initial_balance` menggeser saldo terkini (bukti saldo dihitung ulang, bukan tersimpan)**		HTTP 302; prop: `init=2000000 bal=2300000`	Pass
N06	**Endpoint tab Transaksi (JSON murni, bukan Inertia)**		HTTP 200, `Content-Type` JSON. `total=2`, item berisi `category.label="MAKAN MINUM"` + `parent_label="Operational Expenses"`. Terbukti route `/bank-accounts/transactions` tidak dibaca sebagai ID akun	Pass
N07	**Endpoint tab Pembayaran Invoice**		HTTP 200 `{"data":[],...,"total":0}`	Pass
N08	**Filter tab Transaksi: tipe, pencarian, bulan**		debit → `total=4` semua `debit`; search → `total=1` (`QA Pemasukan`); `month=2026-08` → `total=5`; `month=2026-01` → `total=0`	Pass

ID	SKENARIO	PERCOBAAN	CATATAN: KENAPA GAGAL & APA PERBAIKANNYA	STATUS
N09	**Statistik & breakdown kategori pada detail akun**		`period={"start":"2026-08-01","end":"2026-08-31","label":"Agustus 2026","is_all_time":false}`; `stats={"total_income":500000,"total_expense":866000,"net_cashflow":-366000,"transaction_count":5}`	Pass
N10	`overallSummary` **lintas akun**		`{"total_balance":1634000,"income":500000,"expense":866000}` — cocok: akun 1 = 2.000.000 + 500.000 – 866.000 = 1.634.000; akun 2 = 0	Pass
N11	**Export PDF laporan kas (rentang tanggal)**		HTTP 200, `Content-Type: application/pdf`, `filename=cash-flow-2026-08-01-to-2026-08-31-account-1.pdf`	Pass
N12	**Alias** `date_from`**/**`date_to` **diterima** `parseFilters()`		Nama file identik: `cash-flow-2026-08-01-to-2026-08-31-account-1.pdf`	Pass
N13	**Export mode "semua waktu" + preview inline**		`attachment; filename=cash-flow-semua-waktu.pdf`; preview → `inline; filename=cash-flow-preview.pdf`	Pass
N14	**Cascade delete: hapus rekening ikut menghapus transaksi & payment-nya**		Saldo terbaca 5.300.000 (payment ikut menambah saldo). Setelah `DELETE` (HTTP 302): `bank_accounts id=4` → `0 baris`, `payments bank_account_id=4` → `0`, `bank_transactions bank_account_id=4` → `0`. Cascade terbukti. Namun ada efek samping tak diinginkan → lihat BUG-BA-01	Fail
N15	**Manager (finance manager) boleh CRUD rekening**		POST 302 (akun `QA Manager Acc` terbentuk), DELETE 302 (akun id=3 terhapus)	Pass
N16	**Whitelist sort pada tab Pembayaran**		HTTP 200, respons normal — `match()` whitelist bekerja	Pass
N17	**Saldo rekening terhitung dinamis di layar**		Rp 14.850.000 = 10.000.000 + 5.200.000 - 350.000. Breakdown kategori ikut menyesuaikan hasil pemindahan kategori pada transaction-categories C05	Pass
B. KONDISI TIDAK NORMAL / EDGE CASE				
A01	`account_number` **duplikat saat create**		HTTP 422 `{"account_number":["The account number has already been taken."]}`	Pass
A02	**Semua field wajib dihilangkan total**		HTTP 422 `{"account_name":["The account name field is required."],"account_number":[...required],"bank_name":[...required],"initial_balance":[...required]}`	Pass
A03	`initial_balance` **di bawah batas (**`min:0`**)**		HTTP 422 `{"initial_balance":["The initial balance field must be at least 0."]}`	Pass
A04	**Update memakai nomor rekening milik akun lain**		HTTP 422 `{"account_number":["The account number has already been taken."]}`	Pass
A05	**Update rekening yang tidak ada**		HTTP 404	Pass
A06	**Tab transaksi tanpa parameter** `account`		HTTP 200 `{"data":[],"total":0,"current_page":1,...}`	Pass
A07	**Tab transaksi dengan** `account` **yang tidak ada**		HTTP 200 `total=0`	Pass

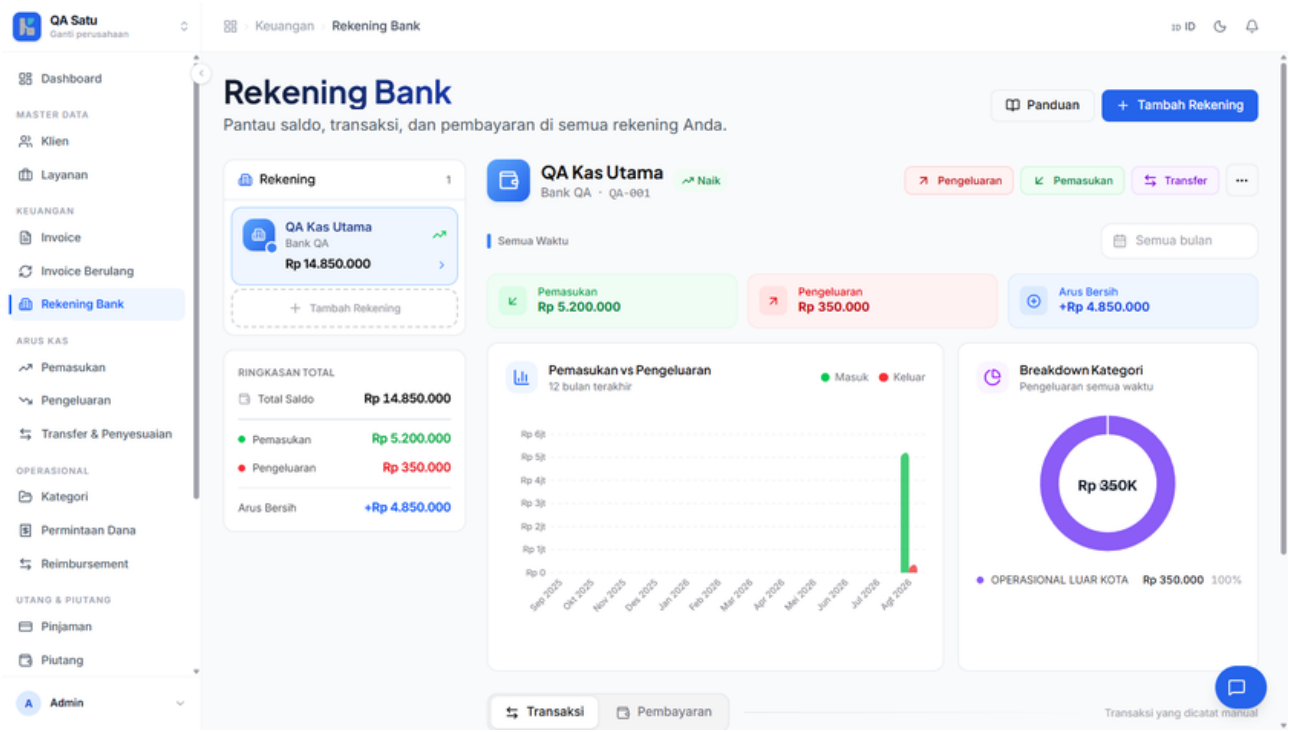
ID	SKENARIO	PERCOBAAN	CATATAN: KENAPA GAGAL & APA PERBAIKANNYA	STATUS
A08	**Staff (tanpa `view bank-accounts`) buka halaman index lewat URL langsung**	🟩	HTTP 403	Pass
A09	**Staff memanggil endpoint mutasi langsung**	🟩	POST 403, DELETE 403	Pass
A10	**Staff memanggil endpoint tab JSON & export PDF langsung**	🟩	keduanya HTTP 403	Pass
A11	**Parameter `sort` tab Transaksi tidak di-whitelist**	🟥🟩	GAGAL pada percobaan pertama: 500 beserta pesan SQL yang menyebut nama database. Setelah BUG-BA-02 diperbaiki: kolom urut disaring whitelist 5 kolom, permintaan menjawab 200 dan jatuh ke urutan bawaan `transaction_date`	Pass
A12	**Percobaan injeksi lewat `sort`**	🟥🟩	GAGAL pada percobaan pertama karena akar yang sama. Setelah diperbaiki: 200	Pass
A13	**Parameter `direction` tidak divalidasi**	🟥🟩	GAGAL pada percobaan pertama: `direction` sembarang juga menjatuhkan endpoint. Setelah diperbaiki: hanya `asc`/`desc` yang diterima, selebihnya jatuh ke `desc`; 200	Pass
A14	`per_page` **ekstrem**	🟥🟩	GAGAL pada percobaan pertama: `per_page=100000` menarik seluruh riwayat. Setelah BUG-BA-03 diperbaiki: dijepit ke 100 (dibuktikan dari nilai `per_page` pada respons)	Pass
A15	**Hapus rekening yang punya `loan_payments` (FK tanpa cascade)**	🟥🟩	GAGAL pada percobaan pertama: 500 mentah dari pelanggaran FK. Setelah BUG-BA-04 diperbaiki: penghapusan ditolak lebih dulu dengan pesan yang menyebut jumlah pembayaran pinjaman penahannya	Pass
A16	**Export PDF untuk `bank_account_id` yang tidak ada**	🟩	HTTP 200, PDF tetap dibuat: `filename=cash-flow-semua-waktu-account-999.pdf`. Tidak crash, tapi memberi laporan untuk rekening yang tak ada	Pass
A17	**Helper picker `/api/bank-accounts` untuk user tanpa `view bank-accounts`**	🟥🟩	GAGAL pada percobaan pertama: endpoint tidak digate. Setelah BUG-BA-05 diperbaiki: digate `can:view bank-accounts`, sehingga user tanpa izin itu menerima 403 — konsisten dengan halaman modulnya	Pass
C. ALUR BERANTAI				
C01	**Pembayaran invoice ikut menaikkan saldo rekening & muncul di tab Pembayaran**	🟩	Saldo akun 4: 5.000.000 − 100.000 + 400.000 = 5.300.000 (cocok). Tab: `{"invoice_number":"001/INV/QD-UB/II/2026","invoice_status":"partially_paid","client_name":"PT Uji Berulang","amount":400000}`	Pass
C02	**Hapus rekening → payment ikut hilang, status invoice tidak dihitung ulang**	🟥🟩	GAGAL pada percobaan pertama: menghapus rekening mengosongkan tabel `payments` tanpa memperbaiki status invoice. Setelah BUG-BA-01 diperbaiki: penghapusan ditolak selama masih ada transaksi, pembayaran invoice, atau pembayaran pinjaman — pesannya menyebut apa saja yang menahan. FK `payments.bank_account_id` juga diubah menjadi `restrictOnDelete` (migration modul Clients), sehingga penjagaan berlaku pula di luar controller	Pass
C03	**Transaksi bank yang memakai kategori bertipe salah tetap ikut dihitung sebagai pengeluaran**	🟥🟩	GAGAL pada percobaan pertama (turunan C02). Setelah diperbaiki: rantai penghapusan itu tidak lagi terjadi, sehingga status invoice dan saldo tetap konsisten	Pass

ID	SKENARIO	PERCOBAAN	CATATAN: KENAPA GAGAL & APA PERBAIKANNYA	STATUS
C04	**Reassign kategori memindahkan angka di breakdown rekening**	■	Sebelum: `QA Induk Beban 111.000`. Sesudah: `QA Induk Beban 166.000` (= 111.000 + 55.000). Perpindahan konsisten. (Bahwa targetnya boleh kategori *parent* adalah masalah tersendiri → BUG-TC-03)	Pass

Bukti visual langkah per langkah

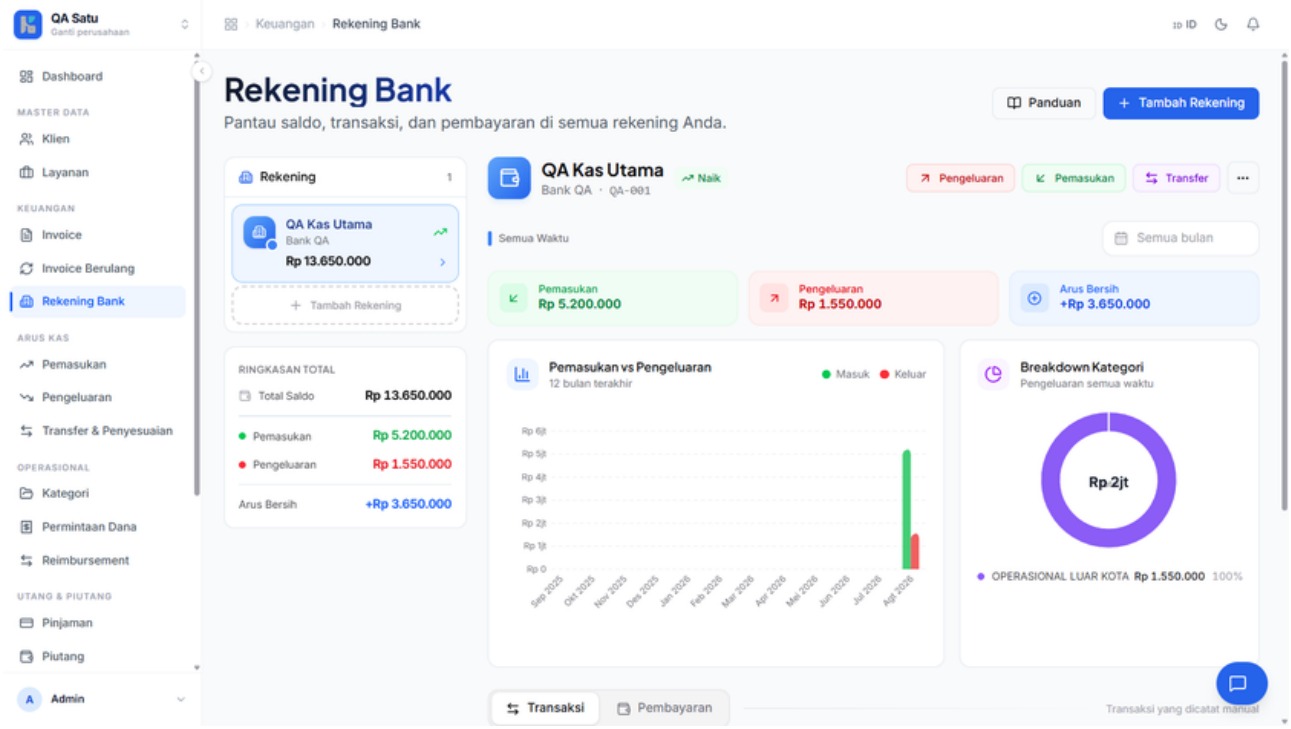
Setiap test case direkam per langkah, bukan hanya hasil akhirnya, supaya alurnya bisa ditelusuri ulang oleh orang lain.

N17 · Saldo rekening terhitung dinamis di layar — Pass



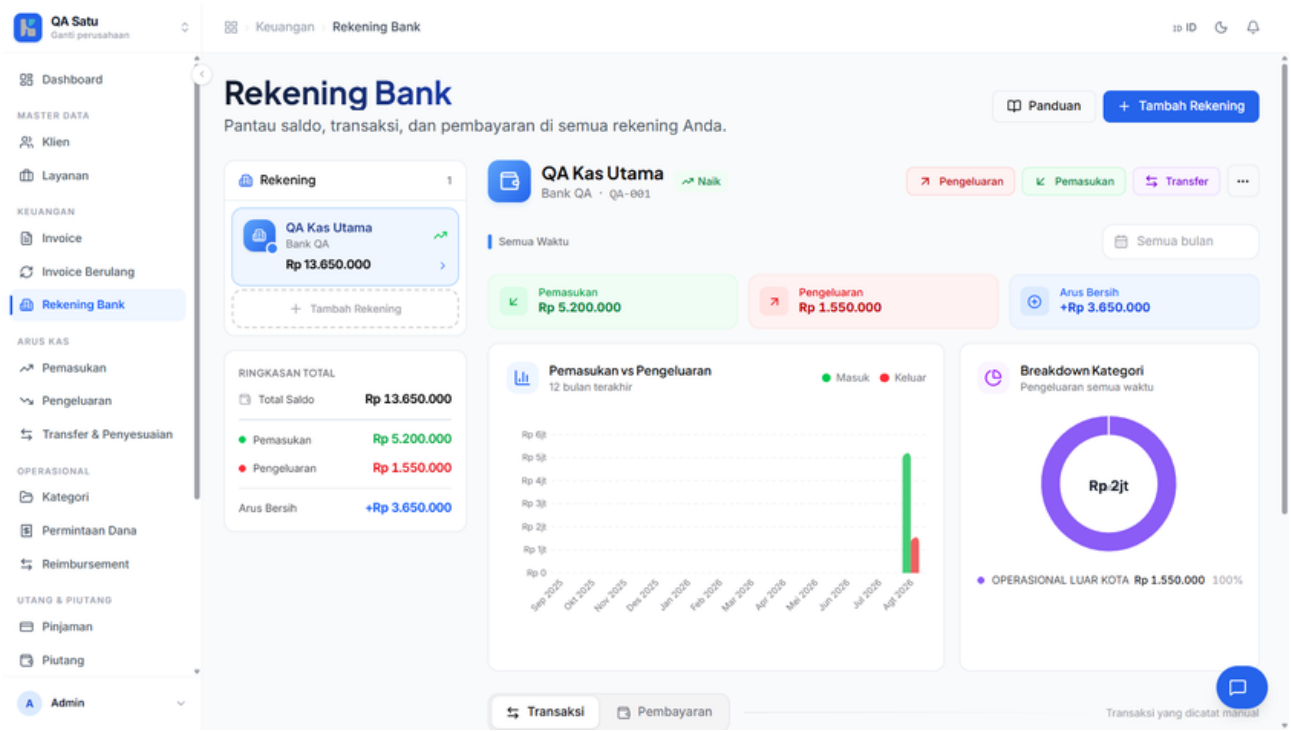
1. Saldo Rp 14.850.000 dihitung ulang dari saldo awal + pemasukan – pengeluaran, bukan disimpan.

A11 · Parameter `sort` tab Transaksi tidak di-whitelist — Pass (percobaan ke-2)



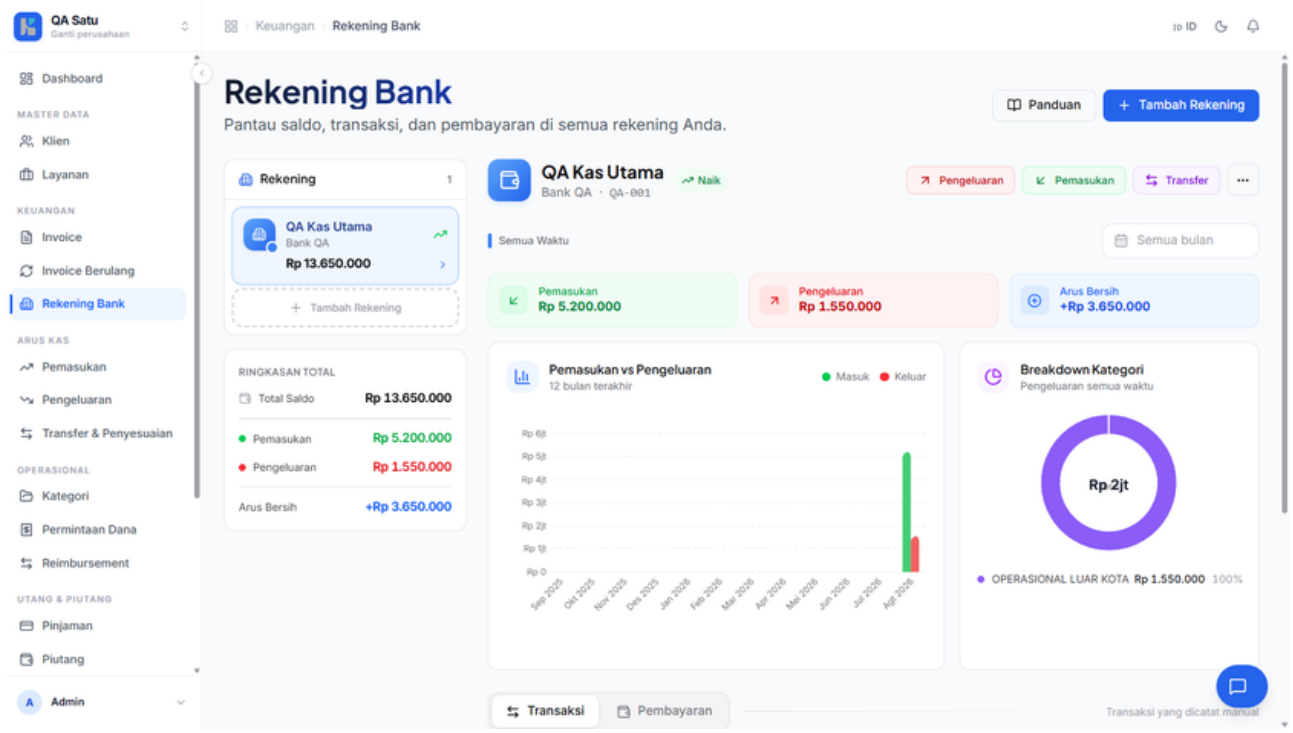
1. Halaman Rekening Bank tetap normal; permintaan tab Transaksi dengan kolom urut sembarang menjawab 200 (dulu 500 dengan pesan SQL yang menyebut nama database).

A12 · Percobaan injeksi lewat `sort` Pass (percobaan ke-2)



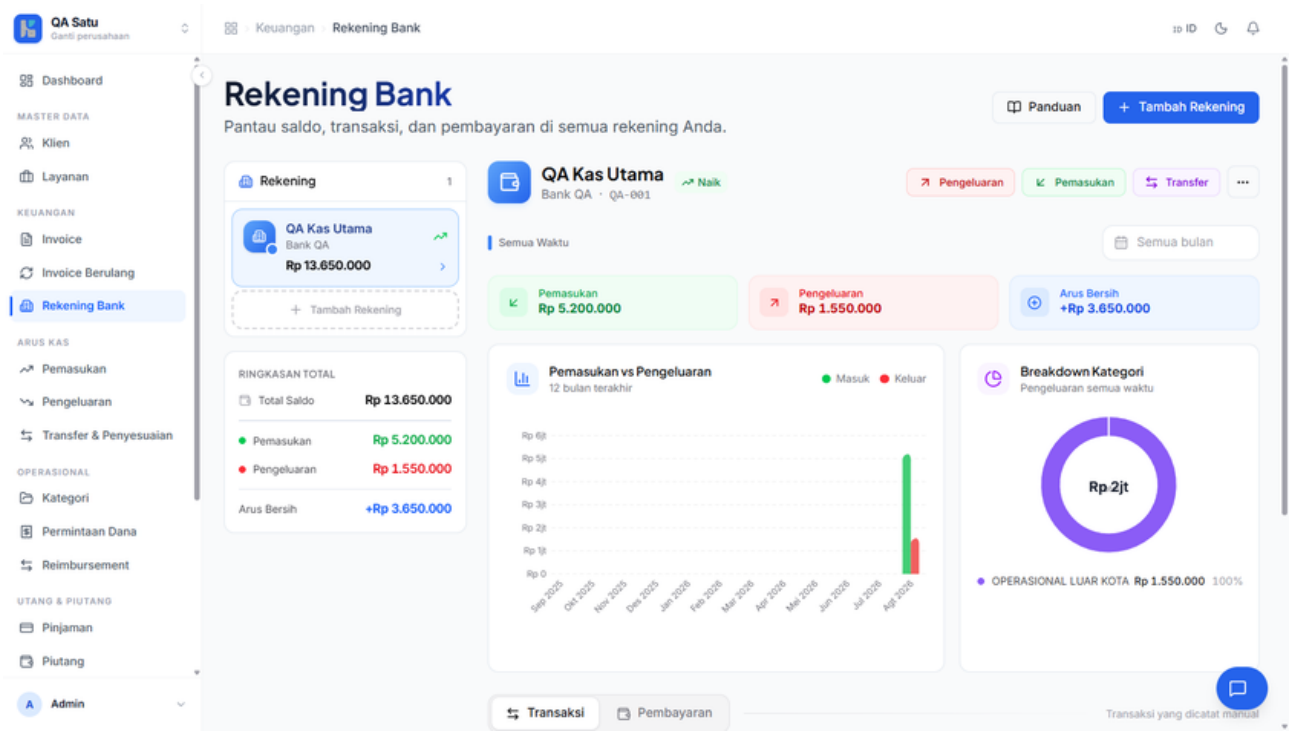
1. Kolom urut di luar whitelist jatuh ke transaction_date alih-alih menjatuhkan endpoint.

A13 · Parameter `direction` tidak divalidasi — Pass (percobaan ke-2)



1. Arah urut di luar asc/desc jatuh ke desc; endpoint tetap 200.

A14 · `per_page` ekstrem — Pass (percobaan ke-2)



1. per_page=100000 dijepit menjadi 100 pada respons endpoint transaksi.

Catatan tambahan

Ditemukan di luar cakupan test case, tapi layak ditindaklanjuti.

Catatan — Catatan pelaksanaan

Cakupan: 16 case Normal (N01–N16), 17 case Abnormal (A01–A17), 4 case Skor: Pass 29, Fail 8 (A11, A12∗, A13, A14, A15, A17, C02, C03). Semua case dieksekusi lewat HTTP dengan sesi login sungguhan, bukan lewat Verifikasi database dilakukan pada setiap case yang mengubah data, Invarian utama modul terbukti benar: (1) saldo tidak pernah disimpan — Kegagalan tooling saya sendiri (bukan cacat aplikasi): percobaan pertama Di luar cakupan modul ini tapi ditemukan saat menyiapkan data: Data uji ditinggalkan di `tenant_qa-dua` (tidak dibersihkan, agar temuan

Kendala alat uji selama pengujian

Dicatat supaya jelas mana yang cacat aplikasi dan mana yang bukan. Tidak satu pun dari ini dihitung sebagai kegagalan test case:

• Berkas respons bawaan helper sempat dipakai bersama beberapa agent sehingga satu agent membaca respons agent lain; helper dipindah ke berkas per-sesi sebelum satu pun hasil dicatat.

• PHP di Windows tidak memetakan /tmp seperti Git Bash, sehingga beberapa skrip pembantu gagal membaca berkas dan diulang dengan path absolut.

Catatan kejujuran

Sebagian besar kode yang diuji ditulis oleh asisten yang sama yang menjalankan pengujian, sehingga ada risiko pengujian mewarisi asumsi yang sama dengan asumsi saat menulis kodenya. Risiko itu terbukti nyata dalam sesi ini: perbaikan lampiran sempat dinyatakan selesai padahal verifikasinya menguji route-nya, bukan URL yang benar-benar dihasilkan aplikasi. Yang sebaiknya diperiksa manusia secara independen: unggah bukti bayar lewat antarmuka lalu pastikan gambarnya benar-benar tampil di layar, bukan sekadar tersimpan dengan path yang benar.

Lingkungan uji: branch multi-tenancy, APP_ENV=local, database pusat finance_central + satu database per perusahaan (tenant_*), aplikasi di http://127.0.0.1:8000. Bukan production..